RELATÓRIO TÉCNICO · SEGURANÇA · DISTRIBUIÇÃO

Router Pro PHP

Testes de funcionamento, análise de vulnerabilidades e guia passo a passo para publicar a biblioteca e permitir que outros usuários a instalem.

Projeto analisado: C:\wamp\www\rotas
Pacote: vendor/router-pro (nome provisório)
Requisito: PHP >= 8.2 · Licença MIT
Data do relatório: 28/10/2026
Ambiente de execução: PHP 8.4.0 · Composer 2.8.6 · PHPUnit 11.5.57 · Windows
Classificação: uso interno / revisão antes de publicação pública

1. Resumo executivo

A biblioteca funciona nos cenários cobertos pelos testes existentes, mas **não está pronta para publicação pública sem correções**. A suíte atual passa (25 testes, 43 asserções), porém há falhas de segurança relevantes que não são detectadas por ela, além de 5 depreciações no PHP 8.4 e 77 erros no PHPStan em nível máximo quando a configuração é corrigida.

25/25	43	5	77	0
TESTES PASSANDO	ASSERÇÕES	DEPRECIAÇÕES PHP 8.4	ERROS PHPSTAN (MAX)	ADVISORIES NO COMPOSER

Bloqueadores antes de publicar (P0): falsificação de IP em `Request::getIp()`; perda de parâmetros de middleware (ex.: `role:admin`); confusão de parâmetros de rota quando o `where()` contém grupo de captura; pilha de grupos que vaza prefixo/middleware após exceção. Esses itens podem causar bypass de autorização, rate limit, auditoria ou roteamento.

Mensagem principal: o componente é uma boa base didática e funcional, mas a camada de segurança ainda depende de a aplicação validar tudo manualmente. Antes do Packagist, corrija os P0, adicione testes de regressão para cada achado e documente claramente os limites da biblioteca.

Escopo e método

- Revisão manual de todos os arquivos de `src/`, testes, documentação, `composer.json` e configurações.
- Execução de `phpunit`, `phpstan` (com configuração corrigida em diretório temporário), `composer validate --strict` e `composer audit`.
- Provas de conceito (PoC) executadas contra a API pública da biblioteca, sem alterar o código-fonte.
- Análise de configuração de publicação: nome do pacote, versionamento, CI, Packagist, distribuição privada e hardening de servidor.

Limite: este trabalho é uma revisão de código e testes direcionados, não um pentest completo nem uma auditoria formal de conformidade. Nenhuma exploração foi feita contra sistemas reais de terceiros.

2. Resultado dos testes automatizados

Verificação	Resultado	Observação
PHPUnit (`vendor/bin/phpunit --testdox`)	PASSOU	25 testes, 43 asserções. Nenhuma falha.
Depreciações PHP 8.4	5	`Request::createFromGlobals()` usa parâmetros implicitamente nullable (`array $x = null`). Corrigir para `?array $x = null`.
PHPStan nível máximo	77 erros	O `phpstan.neon` do repositório é inválido porque `paths:` está fora de `parameters:`. Com a configuração corrigida, o nível max reporta 77 erros (tipos de array, `mixed`, propriedade não lida, etc.).
`composer validate --strict`	VÁLIDO	Metadados básicos válidos, mas o nome `vendor/router-pro` é provisório e faltam `homepage` / `support`.

Verificação	Resultado	Observação
`composer audit --locked`	**LIMPO**	Nenhum advisory conhecido nas dependências de desenvolvimento travadas.
Teste de path traversal existente	**FRÁGIL**	Ele passa porque a URI é reescrita para `/` e não existe rota `/` no teste. Isso não prova que o traversal foi bloqueado.

Correção imediata da configuração do PHPStan: o arquivo atual tem `paths:` na coluna zero. O correto é:

```
parameters:
    level: max
    paths:
        - src
        - tests
```

3. Matriz de vulnerabilidades e riscos

ID	Severidade	Achado	Local principal
SEC-01	**Crítica**	IP spoofing via `X-Forwarded-For`; lista de proxies confiáveis é ignorada	`src/Request.php:163-175`
SEC-02	**Crítica**	Parâmetros de middleware (`role:admin`) são descartados	`src/Middleware/MiddlewareResolver.php:27-43`
SEC-03	**Crítica**	Grupo de captura em `where()` desloca parâmetros da rota	`src/RouteMatcher.php:124-153`
SEC-04	**Alta**	Pilha de grupos não é limpa em exceção e é estática entre instâncias	`src/Router.php:20,151-175`
SEC-05	**Alta**	Normalização de URI divergente e proteção de traversal incompleta/contornável	`src/Request.php:67-83` ; `src/RouteMatcher.php:50-64`
SEC-06	**Alta**	Prefixo de nome de grupo renomeia rotas anteriores e conflitos de nome passam	`src/Router.php:158-174` ; `src/RouteCollection.php:16-46`
SEC-07	**Alta**	Modo debug vaza mensagem e stack trace; pode expor segredos	`src/Router.php:31-36,288-291` ; `src/Exception/Handler.php:15-17`
SEC-08	**Alta**	Partes estáticas da rota não são escapadas; regex do `where()` não é validada	`src/RouteMatcher.php:86-153`
SEC-09	**Média**	URI/parâmetros aceitam newline, NUL e `+` ; `$` aceita newline final	`src/Request.php:69` ; `src/RouteMatcher.php:105,138`
SEC-10	**Média**	Middleware é instanciado sem DI; alias desconhecido gera 500	`src/Middleware/MiddlewarePipeline.php:22-30`
SEC-11	**Média**	Respostas sem headers de segurança/Content-Type padrão; redirect externo sem validação	`src/Response.php:30-33,86-90,171-182`
SEC-12	**Média**	JSON com `JSON_UNESCAPED_SLASHES` e sem flags de escape para contexto HTML	`src/Response.php:43-51`
SEC-13	**Média**	Fallback registrado como rota real `/__fallback__` ; acessível diretamente	`src/Router.php:103-107,227-234`

ID	Severidade	Achado	Local principal
SEC-14	Média	Corpo/URI sem limite e regex recompilada por rota/request (DoS)	`src/Request.php:55` ; `src/RouteMatcher.php:13-25`
SEC-15	Média	ControllerResolver instancia qualquer classe e a interface de container não existe	`src/Controller/ControllerResolver.php:17-90`
SEC-16	Baixa	Depreciações PHP 8.4, URL generator, HEAD/OPTIONS, estado compartilhado	Vários arquivos

4. Achados detalhados

SEC-01 · IP spoofing em `Request::getIp()` Crítica

Arquivo: `src/Request.php:163-175`

O que acontece: se o array `$trustedProxies` não estiver vazio, o método lê `X-Forwarded-For` e retorna o **primeiro** valor da lista. Ele não verifica se `REMOTE_ADDR` é realmente um proxy confiável e não valida o formato do IP. Qualquer cliente pode enviar `X-Forwarded-For: 1.2.3.4` e ser tratado como esse IP.

Evidência (PoC): com `REMOTE_ADDR = 10.0.0.5` e `X-Forwarded-For: 1.2.3.4`, `getIp(['10.0.0.5'])` retornou `1.2.3.4`.

Impacto: bypass de rate limiting, allowlist/denylist por IP, auditoria e controles de acesso; falsificação de logs. É especialmente grave em APIs e painéis administrativos.

Correção recomendada: só confiar em XFF quando `REMOTE_ADDR` estiver na lista; percorrer a cadeia da direita para a esquerda removendo proxies confiáveis; validar com `filter_var(..., FILTER_VALIDATE_IP)` ; documentar que a lista deve vir de configuração, nunca do usuário.

```
// Exemplo de direção segura (adaptar à sua arquitetura)
$remote = $this->server['REMOTE_ADDR'] ?? null;
if ($remote === null || !filter_var($remote, FILTER_VALIDATE_IP)) {
    return null;
}
if (!in_array($remote, $trustedProxies, true)) {
    return $remote; // não é proxy confiável: ignora XFF
}
$xff = $this->getHeader('X-Forwarded-For');
if ($xff === null) {
    return $remote;
}
$ips = array_reverse(array_map('trim', explode(',', $xff)));
foreach ($ips as $ip) {
    if (!filter_var($ip, FILTER_VALIDATE_IP)) {
        continue;
    }
    if (!in_array($ip, $trustedProxies, true)) {
        return $ip;
    }
}
return $remote;
```

SEC-02 · Parâmetros de middleware são descartados Crítica

Arquivos: `src/Middleware/MiddlewareResolver.php:27-43`, `src/Middleware/MiddlewarePipeline.php:22-30`

O que acontece: o resolver reconhece `role:admin`, separa `$name` e `$params`, mas devolve apenas a classe do alias. O `$params` é descartado. O pipeline então faz `new $middleware()` sem argumentos. Um middleware que deveria receber `admin` recebe `null`.

Evidência (PoC): `resolve(['role:admin'])` retornou `['RoleMiddleware']` e o middleware recebeu `null` em vez de `'admin'`.

Impacto: bypass de autorização quando a aplicação usa middleware parametrizado (ex.: `role:admin`, `throttle:60`, `can:edit`). O código parece correto, mas a restrição não é aplicada. Também há falha de disponibilidade: alias com parâmetro desconhecido vira string literal e gera 500.

Correção recomendada: propagar os parâmetros para o middleware (construtor ou método `handle/process`), resolver aliases antes de instanciar e falhar no registro da rota (não em produção) quando o alias não existir. Prefira um objeto de definição ou closure resolvida.

```
// Exemplo: resolver para uma closure que injeta os parâmetros
$class = $aliases[$name] ?? null;
if ($class === null) {
    throw new InvalidArgumentException("Middleware não registrado: {$name}");
}
$args = $params === '' ? [] : explode(',', $params);
$resolved[] = static function (Request $request, callable $next) use ($class, $args) {
    $middleware = new $class(...$args);
    return $middleware->process($request, $next);
};
```

Atenção: o pipeline atual só aceita string/objeto/callable; a closure acima exige que ele trate callables com a assinatura `(Request, callable)` — o que já é suportado.

SEC-03 · Confusão de parâmetros com grupos de captura no `where()` Crítica

Arquivo: `src/RouteMatcher.php:124-153`

O que acontece: o matcher monta a regex da rota e mapeia os parâmetros por posição (`$matches[1]` , `$matches[2]` , ...). Se o regex informado em `where()` tiver um grupo de captura — por exemplo `([0-9]+)` em vez de `[0-9]+` — os índices deslocam e o valor de um parâmetro é atribuído a outro.

Evidência (PoC): rota `/u/{a}/{b}` com `where('a', '([0-9]+)')` ; requisição `/u/12/segredo` retornou `a=12 b=12` (esperado `b=segredo`).

Impacto: o controller recebe o ID/valor errado. Se o parâmetro for usado para autorização (ex.: `conta_id` , `user_id`) ou para consultas, pode ocorrer acesso indevido, alteração do registro errado ou vazamento de dados.

Correção recomendada: usar grupos nomeados (`(?P<nome>...)`) e extrair por nome; validar/rejeitar regex de `where()` que contenham grupos de captura ou o delimitador `#` ; nunca confiar na posição dos grupos. Adicionar teste de regressão com dois parâmetros e regex com grupo.

```
// Direção: nomear os grupos e ler por nome, não por índice
$regex = preg_replace_callback(
    '#\{([a-zA-Z_][a-zA-Z0-9_-]*)(\?)?\}#',
    static function (array $m) use ($route): string {
        $name = $m[1];
        $re = $route->getWheres()[$name] ?? '[^/]+';
        if (preg_match('/\((?!\?)/', $re)) {
            throw new InvalidArgumentException("where({$name}) não pode conter grupos de captura");
        }
        return '/(?P<' . $name . '>' . $re . ')';
    },
    $pattern
);
// ... preg_match e extração via $matches[$name]
```

SEC-04 · Pilha de grupos vaza após exceção e é compartilhada entre instâncias Alta

Arquivo: `src/Router.php:20,151-175`

O que acontece: `group()` empilha atributos, chama o callback e só desempilha no fluxo normal. Se o callback lançar exceção, o `array_pop` não executa. Além disso, `$currentGroupStack` é `static`, então o estado é compartilhado entre instâncias do `Router` no mesmo processo (relevante em Swoole, RoadRunner, FrankenPHP e testes).

Evidência (PoC): após um grupo `admin` com `middleware = ['auth']` lançar exceção, a rota seguinte `/publico` passou a responder 404 em `/publico` e 500 em `/admin/publico` — prefixo e middleware vazaram para a rota seguinte.

Impacto: rotas criadas depois de uma exceção herdam prefixo/middleware indevidos; em servidores de longa duração, uma requisição pode contaminar outra. Pode causar indisponibilidade, roteamento errado e falhas de autorização (middleware inesperado ou ausente).

Correção recomendada: usar `try/finally` para desempilhar; remover o estado `static` ou isolá-lo por instância/request; nunca usar `static` mutável em biblioteca concorrente. Adicionar teste que lança exceção dentro do grupo e verifica a rota seguinte.

```
$this->groupStack[] = $attributes;
try {
    $callback($this);
} finally {
    array_pop($this->groupStack);
}
```

SEC-05 · Normalização de URI divergente e traversal contornável Alta

Arquivos: `src/Request.php:67-83`, `src/RouteMatcher.php:50-64`

O que acontece: a URI é decodificada com `urldecode()` (não `rawurldecode()`), barras invertidas viram barras normais, múltiplas barras são colapsadas e `/../` é substituído por `/`. Isso faz a biblioteca enxergar um caminho diferente do servidor web/WAF. A proteção só detecta `/../`; um `..` final, dupla codificação e newline passam.

Evidências (PoC):

- `/admin%2Fsecret`, `/admin//secret` e `/admin\secret` chegaram à rota `/admin/secret` (200).
- `/files/..` chega ao parâmetro como `path=..`; `/files/..%252F..%252Fetc%252Fpasswd` chega como `..%2F..%2Fetc%2Fpasswd` (se a aplicação decodificar de novo, vira traversal).
- `/public/..%2F..%2Fetc%2Fpasswd` é reescrito para `/` e retorna a home (200), em vez de 404/400.
- `/files/a+b` vira `a b` porque `urldecode()` trata `+` como espaço em path.

Impacto: bypass de regras de WAF/proxy/ACL baseadas no caminho bruto; confusão de roteamento; se o parâmetro for usado em filesystem, include, download ou redirect, pode virar path traversal real. A reescrita silenciosa para `/` também mascara ataques e serve conteúdo inesperado.

Correção recomendada: definir uma política única de canonicalização; usar `rawurldecode()` uma única vez; rejeitar (400/404) segmentos `.`/`..`, barras codificadas, barras invertidas, NUL e caracteres de controle; não reescrever silenciosamente para `/`; garantir que o servidor e o WAF usem a mesma regra. Se a aplicação usar o parâmetro em filesystem, validar com `realpath()` e confinar a um diretório base.

```
$uri = rawurldecode($uri);
if (preg_match('#(?:^|/)\.\.?(?:/|$)#', $uri)
    || preg_match('#%2f|%5c#i', $uri)
    || preg_match('/[\x00-\x1F\x7F]/', $uri)) {
    throw new NotFoundHttpException('URI inválida');
}
```

SEC-06 · Prefixo de nome de grupo corrompe rotas e conflitos não são detectados Alta

Arquivos: `src/Router.php:158-174`, `src/RouteCollection.php:16-46`

O que acontece: ao fechar um grupo com `as`, o `Router` percorre **todas** as rotas já registradas e renomeia qualquer nome que não comece com o prefixo. Isso atinge rotas criadas antes do grupo. Além disso, o índice de nomes não é atualizado após a renomeação; um conflito posterior passa sem `RouteNameConflictException` e `getByName()` reconstrói o índice com "último vence".

Evidência (PoC): rota `/home` nomeada `home` foi renomeada para `admin.home` por um grupo `as = admin.`. Em outro teste, após renomeação, foi possível registrar duas rotas com o nome `x.dup` sem exceção; `route('x.dup')` apontou para `/c` (a última), embora `/a` também tivesse o mesmo nome.

Impacto: links/redirects gerados para a rota errada; telas administrativas apontando para caminhos públicos ou vice-versa; quebra de navegação e possível confusão de autorização se a URL for usada como controle.

Correção recomendada: aplicar prefixo apenas às rotas criadas dentro do callback (guardar o índice inicial); atualizar o índice de nomes após renomear; lançar exceção em duplicidade; evitar `ReflectionProperty` para mutar estado privado — prefira métodos internos. Adicionar testes para rota anterior, grupo aninhado e nome duplicado.

```
$start = count($this->routes->getRoutes());
try {
    $callback($this);
} finally {
    array_pop($this->groupStack);
}
for ($i = $start; $i < count($this->routes->getRoutes()); $i++) {
    // renomear apenas as rotas criadas neste grupo
}
$this->routes->rebuildNameIndex(); // e lançar em duplicidade
```

SEC-07 · Modo debug expõe stack trace e mensagens Alta

Arquivos: `src/Router.php:31-36,288-291`, `src/Exception/Handler.php:15-17`

O que acontece: `new Router(true)` ou `setDebug(true)` faz o handler devolver `$e->getMessage()` e `getTraceAsString()` ao cliente. Em produção, isso pode expor caminhos, nomes de classes, SQL, tokens e outras informações.

Evidência (PoC): exceção com mensagem `senha=segredo` retornou a mensagem e o stack trace completos.

Impacto: vazamento de informações sensíveis; facilita reconhecimento para ataques; em alguns casos, exposição de dados de requisição.

Correção recomendada: padrão `false`; habilitar somente por variável de ambiente explícita (ex.: `APP_DEBUG=1`) e nunca em produção; registrar a exceção no servidor com ID de correlação e devolver mensagem genérica ao cliente. Não incluir corpo/headers em logs sem sanitização.

SEC-08 · Regex da rota não escapa partes estáticas e não valida `where()` Alta

Arquivo: `src/RouteMatcher.php:86-153`

O que acontece: a URI da rota é usada diretamente como regex; apenas `/` é escapado. Pontos, `+`, parênteses, `#` e outros metacaracteres do caminho viram operadores de regex. O `where()` aceita qualquer regex, inclusive com grupos, delimitador `#` e backtracking catastrófico. Erros de compilação são silenciados com `@preg_match`.

Evidência (PoC): rota estática `/files/report.txt` respondeu 200 para `/files/reportXtxt` (o `.` casou com `X`).

Impacto: uma rota pode capturar caminhos que não deveria (route shadowing), confundindo controles de acesso e validações; regex inválida pode derrubar/ocultar rotas; regex maliciosa ou mal escrita pode causar ReDoS e indisponibilidade.

Correção recomendada: compilar a regex no registro da rota, escapando cada trecho estático com `preg_quote($trecho, '#')`; escapar o delimitador; validar o `where()` no registro (sem `#`, sem grupos de captura, comprimento limitado); usar âncora `\z` em vez de `$`; remover `@` e tratar erro de regex; limitar o tamanho da URI; considerar cache de rotas compiladas.

SEC-09 · Newline, NUL e `+` em URI/parâmetros Média

Arquivos: `src/Request.php:69`, `src/RouteMatcher.php:105,138`

O que acontece: `urldecode()` decodifica `%0A`, `%00` e transforma `+` em espaço. A regex usa `$`, que em PCRE casa antes de um newline final. Assim, `/admin/secret%0A` casa com a rota `/admin/secret`, e parâmetros podem conter newline/NUL.

Evidência (PoC): `/admin/secret%0A` retornou 200; `/files/abc%0Adef` e `/files/abc%00def` chegaram ao parâmetro com os bytes de controle; `/files/a+b` virou `a b`.

Impacto: bypass de regras de WAF/allowlist por newline; injeção de log; se o valor for usado em header, e-mail, HTML ou filesystem, pode facilitar header injection, log forging ou comportamento inesperado.

Correção recomendada: rejeitar caracteres de controle na URI; usar `rawurldecode()` para path; usar `\z` ou o modificador `D`; validar parâmetros conforme o tipo esperado (ex.: `whereNumber`) e nunca interpolar valores não validados em headers/HTML.

SEC-10 · Middleware sem injeção de dependência e alias inválido vira 500 Média

Arquivo: `src/Middleware/MiddlewarePipeline.php:22-30`

O que acontece: o pipeline faz `new $middleware()`. Middlewares com dependências no construtor falham; aliases não registrados viram string literal e falham com 500; não há integração com o `ControllerResolver`/container.

Impacto: indisponibilidade de rotas; em cenários de fallback mal configurado, a aplicação pode cair em comportamentos não previstos. Não é bypass por si só, mas enfraquece a arquitetura de segurança.

Correção recomendada: resolver middleware via container (PSR-11); validar aliases no registro da rota; falhar cedo; permitir objetos e closures; documentar precedência.

SEC-11 · Respostas sem headers de segurança e redirect sem validação Média

Arquivo: `src/Response.php:30-33,86-90,171-182`

O que acontece: `Response::make()` não define `Content-Type`; não há `X-Content-Type-Options`, CSP, `X-Frame-Options`, `Referrer-Policy` ou `Cache-Control` por padrão. `Response::redirect($to)` aceita qualquer destino, inclusive externo, sem validar CRLF ou same-origin.

Evidência (PoC): `Response::make('<script>...')` retornou headers vazios; `Response::redirect('https://evil.example/phish')` definiu `Location` externo.

Impacto: MIME sniffing/XSS em navegadores antigos; clickjacking; open redirect se o destino vier de entrada do usuário; vazamento de Referer.

Correção recomendada: definir `Content-Type` seguro por padrão; oferecer middleware/helper de headers de segurança; validar `Location` (permitir apenas caminhos relativos ou hosts allowlisted; rejeitar `\r\n`); documentar CSP/HSTS conforme o deploy.

```
$headers = array_merge([
    'X-Content-Type-Options' => 'nosniff',
    'X-Frame-Options' => 'DENY',
    'Referrer-Policy' => 'no-referrer',
], $headers);
```

SEC-12 · JSON sem flags de escape para contexto HTML Média

Arquivo: `src/Response.php:43-51`

O que acontece: `Response::json()` usa `JSON_UNESCAPED_UNICODE | JSON_UNESCAPED_SLASHES`. Se a aplicação embutir a resposta JSON em `<script>` ou usar JSONP, sequências como `</script>` podem quebrar o contexto e permitir XSS.

Impacto: XSS em cenários de embed; hoje o `Content-Type: application/json` reduz o risco, mas não elimina o risco de composição.

Correção recomendada: documentar que o JSON não deve ser interpolado em HTML; para JSONP/embed, usar `JSON_HEX_TAG | JSON_HEX_AMP | JSON_HEX_APOS | JSON_HEX_QUOT`; considerar não usar `JSON_UNESCAPED_SLASHES` por padrão.

SEC-13 · Fallback acessível em `/__fallback__` Média

Arquivo: `src/Router.php:103-107,227-234`

O que acontece: `fallback()` registra uma rota real no caminho `/__fallback__`. Uma requisição direta a esse caminho executa o fallback; o caminho pode colidir com uma rota legítima.

Evidência (PoC): `GET /__fallback__` retornou o conteúdo do fallback com status 404.

Impacto: comportamento inesperado e possível confusão de roteamento; baixo impacto direto, mas é um cheiro de design.

Correção recomendada: não registrar o fallback como rota; guardar o callable em uma propriedade e invocá-lo quando o matcher não encontrar rota (ou quando 404 for apropriado), sem expor um caminho reservado.

SEC-14 · Falta de limites e compilação de regex por requisição Média

Arquivos: `src/Request.php:55`, `src/RouteMatcher.php:13-25`

O que acontece: `file_get_contents('php://input')` lê o corpo inteiro sem limite configurável; a URI não tem limite; a regex é montada para cada rota em cada requisição; `preg_match` roda sobre todas as rotas até achar candidatos. Regex de `where()` com backtracking catastrófico pode consumir CPU.

Impacto: negação de serviço por corpo/URI gigante ou regex custosa; degradação proporcional ao número de rotas.

Correção recomendada: limites configuráveis de URI/corpo (ex.: 414/413), validação de `Content-Length`, cache de rotas compiladas, limites de PCRE (`pcre.backtrack_limit`) e revisão de regex de `where()`. Em produção, configure `client_max_body_size` / `LimitRequestBody`.

SEC-15 · ControllerResolver instancia qualquer classe; container não existe Média

Arquivo: `src/Controller/ControllerResolver.php:17-90`

O que acontece: o resolver aceita qualquer classe existente e qualquer método público; `make()` instancia recursivamente classes tipadas no construtor. A interface `ContainerInterface` é referenciada no namespace `RouterPro\Controller`, mas não existe no projeto, então a integração com container está quebrada. Não há allowlist de namespace/interface de controller.

Impacto: se a ação da rota vier de configuração dinâmica ou entrada não confiável, pode haver instanciação de classes arbitrárias e chamada de métodos públicos indevidos. Mesmo com rotas confiáveis, é uma superfície desnecessária.

Correção recomendada: exigir que o controller implemente uma interface/base ou esteja em namespace allowlisted; usar PSR-11 para container; não instanciar classes fora do namespace da aplicação; validar o método (não permitir magic methods como `__construct`, `__destruct`); documentar que a ação da rota é código confiável.

4.1 Achados adicionais (baixa severidade / qualidade)

- **SEC-16:** depreciações PHP 8.4 em `Request::createFromGlobals()` — trocar `array $x = null` por `?array $x = null`.
- **RouteUrlGenerator:** parâmetro no meio do segmento (ex.: `/files/{name}.json`) gera `//` e não valida contra `where()`; `rawurlencode()` é aplicado, mas o formato final não é canonicalizado.
- **HEAD/OPTIONS:** não há mapeamento automático de HEAD para GET nem resposta automática de OPTIONS; `Response::send()` envia corpo em HEAD.
- **Estado compartilhado:** `Router::$currentGroupStack` é estático e `currentRoute` é mutável; cuidado em servidores de longa duração/corrotinas.
- **CSRF/CORS/rate limit:** não fazem parte da biblioteca; a documentação deve deixar claro que a aplicação precisa implementá-los. Method spoofing deve ser usado com token CSRF.
- **Documentação:** `SECURITY.md` pede e-mail, mas não informa um contato; `docs/instalacao.md` usa nome de pacote provisório; README promete "resolução segura" que não corresponde ao estado atual.
- **Repositório:** `.phpunit.result.cache` está versionado (deveria ser ignorado); `phpstan.neon` inválido; ausência de CI; `composer.lock` presente (ok para dev, mas avalie export-ignore).

5. Plano de correção priorizado

Prioridade	Itens	Critério de saída
P0	SEC-01, SEC-02, SEC-03, SEC-04	Corrigir, adicionar testes de regressão, revisar autorização/rate limit. Não publicar antes.
P1	SEC-05, SEC-06, SEC-07, SEC-08	Canonicalização de URI, nomes de rota, debug seguro e compilação/validação de regex.
P2	SEC-09 a SEC-15	Hardening de headers, JSON, fallback, limites, controller/container e documentação.
P3	SEC-16, qualidade, CI, docs	PHPStan limpo, CI multi-versão, changelog, política de segurança, badges.

> **Definição de pronto para publicação:** P0/P1 corrigidos; suíte com testes de regressão para cada achado; PHPStan nível ≥ 8 sem erros (ou baseline justificada); CI verde em PHP 8.2, 8.3 e 8.4; `composer validate --strict` e `composer audit` limpos; README com avisos de segurança; SECURITY.md com contato real; versão `1.0.0` tagueada.

6. Guia de publicação para outros usuários

Há dois caminhos: **público via Packagist** (recomendado para open source) e **privado via repositório VCS/path/Satis** (para empresas). Abaixo, o fluxo público completo, com alternativas privadas no final.

6.1 Pré-requisitos e decisões

- Conta no GitHub/GitLab e conta no Packagist.
- Nome de pacote definitivo no formato `vendor/package`, em minúsculas. **Não use `vendor/router-pro`**: "vendor" é genérico e não identifica você. Exemplos: `valtercouto/router-pro`, `suaempresa/router-pro`.
- Licença definida (MIT já está no projeto) e titular do copyright atualizado.
- Repositório Git com histórico limpo, branch principal `main` e tags SemVer.
- Decisão sobre suporte: versões suportadas, canal de segurança e política de depreciação.

6.2 Ajustar o `composer.json`

Troque o nome provisório, adicione metadados de suporte e mantenha o autoload PSR-4. Não coloque `version` no `composer.json`: o Packagist deriva a versão das tags Git.

```
{
  "name": "valtercouto/router-pro",
  "description": "Roteador HTTP em PHP puro, inspirado na API do Laravel, com middlewares e grupos.",
  "type": "library",
  "license": "MIT",
  "keywords": ["router", "routing", "http", "middleware", "psr-4", "php"],
  "homepage": "https://github.com/valtercouto/router-pro",
  "support": {
    "issues": "https://github.com/valtercouto/router-pro/issues",
    "source": "https://github.com/valtercouto/router-pro"
  },
  "authors": [
    {"name": "Valter Couto Santos Junior", "role": "Developer"}
  ],
  "require": {
    "php": ">=8.2"
  },
  "require-dev": {
    "phpunit/phpunit": "^11.0",
    "phpstan/phpstan": "^2.2"
  },
  "autoload": {
    "psr-4": {"RouterPro\\": "src/"}
  },
  "autoload-dev": {
    "psr-4": {"RouterPro\\Tests\\": "tests/"}
  },
  "scripts": {
    "test": "vendor/bin/phpunit",
    "stan": "vendor/bin/phpstan analyse --no-progress"
  },
  "config": {"sort-packages": true},
  "minimum-stability": "stable",
  "prefer-stable": true
}
```

Ajuste o nome do vendor para o seu usuário/organização. O nome do pacote no Packagist é imutável; escolha com cuidado. O `phpstan` foi atualizado para ^2.2; se preferir manter 1.x, use uma versão suportada e corrija a configuração.

6.3 Corrigir a configuração do PHPStan

```
parameters:
    level: max
    paths:
        - src
        - tests
```

O arquivo atual está com `paths:` fora de `parameters:`, o que faz o PHPStan abortar com “Found section 'paths' ... extension is missing”. Depois de corrigir, rode `composer stan` e trate os 77 erros (ou reduza o nível com justificativa e crie um baseline).

6.4 Preparar o repositório e o pacote

```
# na raiz do projeto
git init
git add .
git commit -m "chore: versão inicial do Router Pro"
git branch -M main
git remote add origin https://github.com/valtercouto/router-pro.git
git push -u origin main
```

Crie um `.gitattributes` para enxugar o pacote distribuído (o Composer baixa o dist do Git, respeitando export-ignore):

```
/tests export-ignore
/benchmarks export-ignore
/docs export-ignore
/.github export-ignore
/phpunit.xml export-ignore
/phpstan.neon export-ignore
/.gitignore export-ignore
/.gitattributes export-ignore
/composer.lock export-ignore
```

Se quiser manter `docs/` no pacote, remova a linha correspondente. Não envie `.env`, chaves, logs ou `vendor/`.

6.5 Adicionar CI (GitHub Actions)

```yaml
name: CI
on:
  push:
    branches: [main]
  pull_request:

jobs:
  tests:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        php: ['8.2', '8.3', '8.4']
    steps:
      - uses: actions/checkout@v4
      - uses: shivammathur/setup-php@v2
        with:
          php-version: ${{ matrix.php }}
          coverage: none
      - run: composer validate --strict
      - run: composer install --prefer-dist --no-progress
      - run: composer test
      - run: composer stan
      - run: composer audit --locked --no-interaction
```

Adicione o badge no README. O CI deve falhar se os testes, o PHPStan, a validação do Composer ou o audit falharem.

6.6 Versionar e taguear (SemVer)

```
git tag -a v1.0.0 -m "Router Pro 1.0.0"
git push origin main --tags
```

- **MAJOR** (`2.0.0`): quebra de compatibilidade.
- **MINOR** (`1.1.0`): nova funcionalidade compatível.
- **PATCH** (`1.0.1`): correção compatível (inclui segurança).
- Pré-release: `v1.0.0-rc.1`; o Composer não instala pré-release sem `@rc` / `^1.0@rc`.

Atualize o `CHANGELOG.md` antes de cada tag, no formato Keep a Changelog. Para falhas de segurança, publique um patch e um advisory (ver 6.10).

6.7 Publicar no Packagist

1. Acesse packagist.org/packages/submit e informe a URL do repositório Git.

2. Confirme o nome do pacote (ele deve bater com `composer.json`).
3. Configure a atualização automática: instale o Packagist app no GitHub ou adicione o webhook informado. Sem isso, use "Update" manualmente após cada tag.
4. Verifique a página do pacote: README, licença, versões, requisitos e badges.
5. Teste a instalação em um projeto limpo (ver 6.8).

> **Nome do vendor:** o Packagist exibe o pacote como `vendor/package`. Se outra pessoa já controla o vendor, use outro nome. Não tente publicar com o nome provisório `vendor/router-pro`.

6.8 Instalação e uso pelos consumidores

```
# em um projeto consumidor
composer require valtercouto/router-pro:^1.0
```

O autoload PSR-4 já registra `RouterPro\`. Exemplo mínimo:

```
<?php
declare(strict_types=1);

require __DIR__ . '/vendor/autoload.php';

use RouterPro\Router;
use RouterPro\Response;

$router = new Router(debug: false); // nunca true em produção

$router->get('/', fn () => Response::json(['ok' => true]));

$router->get('/usuarios/{id}', [App\Controllers\UsuarioController::class, 'show'])
    ->whereNumber('id')
    ->middleware('auth');

$response = $router->dispatchFromGlobals();
$response->send();
```

Estrutura recomendada: `public/index.php` como único entrypoint; `vendor/` fora do document root; `routes/` e `app/` fora de acesso público.

6.9 Configuração de servidor (produção)

Apache (`public/.htaccess`)

```
RewriteEngine On
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^ index.php [L]

# negar acesso a arquivos sensíveis
<FilesMatch "^\.">
  Require all denied
</FilesMatch>
```

Nginx

```
root /var/www/app/public;
index index.php;

location / {
  try_files $uri $uri/ /index.php?$query_string;
}

location ~ \.php$ {
  include fastcgi_params;
  fastcgi_pass unix:/run/php/php8.4-fpm.sock;
  fastcgi_param SCRIPT_FILENAME
$document_root$fastcgi_script_name;
}

location ~ /\.(?!well-known) { deny all; }
```

- Document root deve ser `public/`, nunca a raiz do projeto.

- Bloqueie `vendor/`, `composer.json`, `.env`, `.git/`, logs e backups.
- HTTPS obrigatório; HSTS após validar; headers de segurança (CSP, nosniff, frame-options).
- `APP_DEBUG=0` em produção; logs fora do document root; rotação e sanitização.
- Limites de corpo/URI no servidor (ex.: `client_max_body_size`, `LimitRequestBody`) para complementar SEC-14.

6.10 Segurança e manutenção pós-publicação

- Substitua o `SECURITY.md` por um contato real (e-mail ou GitHub Security Advisories) e defina versões suportadas.
- Use GitHub Security Advisories para receber relatos privados; não peça para abrir issue pública.
- Publique correções como PATCH/MINOR e descreva o impacto no CHANGELOG sem entregar exploit antes da correção.
- Rode `composer audit` no CI; atualize dependências de desenvolvimento.
- Mantenha uma política de depreciação: anunciar em MINOR, remover em MAJOR.
- Adicione testes de regressão para cada CVE/correção de segurança.
- Documente claramente o que a biblioteca **não** faz: CSRF, CORS, autenticação, rate limit, validação de entrada e headers de segurança.

6.11 Distribuição privada (sem Packagist)

Se o pacote não for público, use uma destas opções no `composer.json` do projeto consumidor:

```
// Repositório VCS privado (GitHub/GitLab)
"repositories": [
  {"type": "vcs", "url": "git@github.com:suaempresa/router-pro.git"}
]

// Pasta local (monorepo/desenvolvimento)
"repositories": [
  {"type": "path", "url": "../router-pro", "options": {"symlink": true}}
]
```

Para várias equipes, considere Satis (estático) ou Private Packagist (com controle de acesso, espelhamento e auditoria). Configure credenciais SSH/tokens no CI, nunca no repositório.

6.12 Checklist final de publicação

- ☐ P0/P1 corrigidos e revisados por outra pessoa.
- ☐ Testes de regressão para SEC-01 a SEC-08 verdes.
- ☐ `composer test`, `composer stan`, `composer validate --strict` e `composer audit` verdes.
- ☐ PHP 8.2, 8.3 e 8.4 testados no CI.
- ☐ `composer.json` com nome definitivo, homepage, support e licença.
- ☐ `phpstan.neon` válido e nível ≥ 8.
- ☐ README com instalação, exemplo, requisitos, segurança e versão.
- ☐ CHANGELOG atualizado; tag `v1.0.0` criada e enviada.
- ☐ SECURITY.md com contato e versões suportadas.
- ☐ LICENSE e copyright corretos; CODE_OF_CONDUCT e CONTRIBUTING revisados.
- ☐ Repositório sem `.env`, chaves, logs, `vendor/` ou cache versionado.
- ☐ Pacote publicado no Packagist e instalado em projeto limpo.
- ☐ Webhook/auto-update do Packagist funcionando após nova tag.
- ☐ Servidor de produção com document root em `public/`, HTTPS, debug off e headers de segurança.

7. Apêndice A — Comandos úteis (PowerShell)

```
# validação local
composer validate --strict
composer install --prefer-dist --no-progress
composer test
composer stan
composer audit --locked --no-interaction

# análise estática direta, ignorando config quebrada
vendor\bin\phpstan analyse src tests --level=max --no-progress

# publicação
git status
git add .
git commit -m "fix: corrige falhas de segurança P0/P1"
git push origin main
git tag -a v1.0.0 -m "Router Pro 1.0.0"
git push origin main --tags
```

8. Apêndice B — Exemplo de middleware parametrizado seguro

```
<?php
declare(strict_types=1);

namespace App\Middleware;

use RouterPro\Request;
use RouterPro\Response;

final class RequireRole
{
    public function __construct(private readonly string $role) {}

    public function process(Request $request, callable $next): Response
    {
        $userRole = $request->getAttribute('user_role'); // ou serviço de auth
        if ($userRole !== $this->role) {
            return Response::text('Proibido', 403);
        }
        return $next($request);
    }
}

// Registro: role:admin deve instanciar RequireRole('admin')
// Se o resolver descartar o parâmetro, a autorização NÃO é aplicada.
```

A biblioteca atual não possui `Request::getAttribute()`; o exemplo ilustra o contrato esperado. Adapte para o mecanismo de autenticação da sua aplicação e nunca confie em role vinda do cliente.

9. Conclusão

O Router Pro tem uma API agradável e cobre bem o roteamento básico, mas a segurança ainda não acompanha a ambição da documentação. Os quatro bloqueadores (IP spoofing, perda de parâmetros de middleware, confusão de parâmetros e vazamento de estado de grupos) precisam ser corrigidos antes de qualquer publicação pública. Depois disso, o caminho para o Packagist é direto: nome definitivo, CI, tags SemVer, metadados completos, política de segurança e testes de regressão.

Publicar é fácil; manter a confiança de quem instala é o trabalho contínuo. Trate cada achado como um teste que deve existir na suíte e cada correção como uma oportunidade de documentar o comportamento esperado.

Relatório gerado em 28/10/2026 a partir da revisão do código em `C:\wamp\www\rotas`. As PoCs foram executadas localmente, sem alterar os arquivos do projeto. Este documento não substitui uma auditoria de segurança independente.